in_class_lab7

Chapter 5 table and data maintenance, then put your work on GitHub

CIS 4720 Database Management and SQL

Points	10 points
Time	About 65 minutes
Software	VS Code with SQLTools, the MySQL on your lab PC, the OneDrive app, and a web browser for GitHub
Starts from	in_class_lab6. You do not need the database you built then, because this lab rebuilds it
You submit	One thing only: the web address of your GitHub repository
Due	The due date is listed on Canvas LMS

Read this first. Why any of this matters.

Everything you create on a lab PC is deleted the same night. The machine is wiped and handed to the next class. The database you built in lab6 is already gone, and so is anything you saved to the desktop or to Documents.

That is not a problem you can work around by being careful. It is why this lab has you save your work in two places that are not the lab PC:

- **OneDrive**, which copies your folder to the cloud as you work, so it is there when you sit at a different machine or open your laptop at home.
- **GitHub**, which keeps your file with a web address you can hand to anyone, which is how you get credit for this lab.

Neither one happens on its own. You have to sign in to OneDrive on the PC before you start, or your folder is just a folder on a machine that gets erased tonight.

> **Sign in to the OneDrive app, not the website**
>
> You can reach OneDrive two ways, through the app installed on the PC or through a browser at office.com. For this lab, use the **app**.
>
> Here is why. When you create a file or a folder in the browser, it exists in the cloud first, and the app on the PC has to notice and pull it down. That can take a while, and sometimes it does not happen until you force it. So you make a folder in the browser, go looking for it in File Explorer to open it in VS Code, and it is not there. Students lose twenty minutes to this every time.
>
> Going the other way round is reliable. Create the folder in File Explorer under your OneDrive, and the app uploads it for you within seconds. You can watch it happen in the system tray.
>
> So the rule for this lab is simple: **make folders and files on the PC, and let OneDrive push them up.** Never make them in the browser and hope they come down.

Part 1. Save your work somewhere that survives tonight

Step 1. Sign in to the OneDrive app

Look in the system tray at the bottom right of the screen for the OneDrive cloud icon. If it is grey or has a line through it, you are not signed in.

- Click the cloud icon. If it asks you to sign in, use your campus email address and password.
- If there is no cloud icon at all, press the Windows key, type **OneDrive**, and open it from the list.
- Wait until the icon turns into a plain cloud or shows a tick. That means it is connected and syncing.

Do not skip this and come back to it later. If OneDrive is not signed in when you create the folder in step 2, nothing in it gets uploaded.

Step 2. Make the folder, in File Explorer

Open File Explorer. In the left sidebar you will see your OneDrive, which on a campus machine is usually named OneDrive followed by the university name. Click it, then make a new folder called:

```
CIS4720_Lab7
```

It has to be inside OneDrive. A folder on the desktop or in Documents is not in OneDrive and will not be uploaded.

Step 3. Download the SQL file into that folder

Download **lab6_backup.sql** from Canvas and save it straight into **CIS4720_Lab7**. If your browser dropped it in Downloads instead, move it across.

Hover over the file in File Explorer and look at the status column. A green tick means OneDrive has it. A pair of circling arrows means it is still uploading, which is fine, just give it a moment.

Part 2. Open it, and learn how to comment

Step 4. Open the folder in VS Code and connect

Open VS Code, then **File**, then **Open Folder**, and choose **CIS4720_Lab7**. Click **lab6_backup.sql** in the Explorer panel on the left to open it.

Then connect to MySQL the same way you did in the midterm lab: the database icon in the left bar, your saved connection, click to connect. If the connection is not there because this is a different PC, add it again with server **localhost**, port **3306**, database **cis4720_yourname**, user **root**.

Step 5. Commenting out one line

A comment is text the database ignores. You use it to explain what a statement does, and to switch a statement off without deleting it.

For a single line, put two dashes and a space at the front:

```
-- This line is a comment. MySQL ignores it completely.
-- SELECT * FROM PRODUCT;    <-- this statement is now switched off

SELECT * FROM PRODUCT;       -- a comment can also sit after a statement
```

> **The fast way, and the one to actually use**
>
> You do not have to type the dashes. Select as many lines as you like, then press **Ctrl and the forward slash key**. VS Code puts **--** at the front of every line you selected. Press it again with the same lines selected and it takes them off.
>
> This is the one to use all through this lab. It works on one line or forty, and it never goes wrong.

Step 6. Commenting out a block, and the trap in it

There is a second way, which wraps a whole region between two markers:

```
/* Everything between these two markers is ignored,
   however many lines it runs across.
   SELECT * FROM PRODUCT;
*/
```

Block comments do not nest, and this will catch you

You cannot put one block comment inside another. SQL does not look for matching pairs. It starts ignoring at the first **/*** and starts paying attention again at the very next ***/** it finds, whatever was in between.

So if you wrap a region that already contains a block comment, the inner ***/** ends your outer comment early. Everything after it runs when you thought you had switched it off, and the leftover ***/** gives you **error 1064, syntax error near '*/'**.

This is why step 9 has you use **Ctrl and forward slash** rather than block markers. Line comments cannot nest wrongly because every line stands on its own.

Part 3. Rebuild the lab6 database

Step 7. Run the file, the first time

Open **lab6_backup.sql** and read the comments at the top. Then run the whole file. In SQLTools, right click in the file and choose **Run Query**, or use the Run link above the statements.

When it finishes you should see four results come back, ending with:

```
products  total_price
       4        63.15

Tables_in_cis4720_yourname  Table_type
PRODUCT                     BASE TABLE
PRODUCT_PRICES              VIEW
```

That is the whole of lab6 rebuilt: the schema, the table, four rows and the view. Whatever happened in lab6, you and everyone else are now in the same place.

Step 8. Run it again on purpose, and read the errors

Run the same file a second time without changing anything. You will get six errors. Do not fix them yet. Read them.

```
ERROR 1050 (42S01): Table 'PRODUCT' already exists
ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'
ERROR 1062 (23000): Duplicate entry '2' for key 'PRIMARY'
ERROR 1062 (23000): Duplicate entry '3' for key 'PRIMARY'
ERROR 1062 (23000): Duplicate entry '4' for key 'PRIMARY'
ERROR 1050 (42S01): Table 'PRODUCT_PRICES' already exists
```

Nothing is broken. Every one of those errors is the database protecting your data:

- **1050** says the table is already there. CREATE TABLE will not quietly replace a table that has your data in it, which is exactly what you would want.

- **1062** says the primary key already has a row with that id. The four INSERT statements are trying to add rows that exist, and the primary key refuses, four times.
- **1050** again for the view, for the same reason as the table.

Step 9. Switch off the statements that only needed to run once

This is the point of steps 7 and 8. CREATE and INSERT build things. Once built, they are built, and running them again can only fail. From here on you want to run this file freely without wading through six errors every time, because by the end of the lab you will be adding to it and running it over and over.

So switch those statements off, without deleting them:

1. Click on the first line of **SECTION 2**, the CREATE TABLE part.
2. Hold **Shift** and click on the last line of **SECTION 4**, the line that reads **FROM PRODUCT;** under the CREATE VIEW. That selects sections 2, 3 and 4 together.
3. Press **Ctrl and the forward slash key**. Every selected line gets **--** in front of it and turns grey.
4. Run the file again. Zero errors, and the four read only checks in SECTION 5 still return their results.

Leave SECTION 5 switched on. Those are SELECT statements, they only read, and they are safe to run as often as you like. Keep the commented out sections in the file. They are how you rebuild from nothing next time you sit at a wiped PC: select them, press **Ctrl and forward slash** again, and they come back to life.

Part 4. Chapter 5, sections 5.1 to 5.6

Six short steps, one for each section of Chapter 5. Add each one to the bottom of **lab6_backup.sql**, under the heading already waiting for it, and run it before you move on.

> **Write a comment above every statement. This is graded.**
>
> Above each statement you add, put a line comment in **your own words** saying what it does. Not a copy of the heading. Something like:
>
> `-- Makes a copy of PRODUCT with only three of the columns`
>
> One point of this lab is for the file to still make sense to you in three weeks, when you have forgotten the lab but still have the file.

Step 10. Section 5.1, look at your schema

A schema is the container holding your tables. These two statements tell you which schema you are in and what is inside it.

```
SELECT DATABASE() AS my_schema;

SHOW FULL TABLES;
```

The second one is worth remembering. Plain **SHOW TABLES** lists your views mixed in with your tables and gives you no way to tell which is which. Adding **FULL** adds a Table_type column that says BASE TABLE or VIEW.

Step 11. Section 5.2, look at the data types

```
DESCRIBE PRODUCT;
```

Compare what comes back against why each type was chosen:

Column	Type	Why that type
prod_id	int	A whole number with no decimal part, so INT
prod_name	varchar(40)	Text of varying length, up to 40 characters
unit_price	decimal(7,2)	Money. DECIMAL keeps exactly 2 decimal places and does not drift the way a floating point type does
added_on	date	A real date, so the database can compare and sort it properly
roast_level	varchar(10)	Short text, added later with ALTER TABLE in lab6

The Null column tells you which columns may be left empty, and the Key column marks the primary key with PRI.

Step 12. Section 5.3, create a table by copying it

You can build a new table out of a SELECT statement instead of listing the columns yourself. The SELECT is the subquery, and the new table gets one column for each column the SELECT returns.

```
CREATE TABLE PRODUCT_COPY AS
SELECT prod_id, prod_name, unit_price
FROM PRODUCT;

SELECT * FROM PRODUCT_COPY;
```

Four rows and three columns, because the SELECT asked for three. Now look at what you actually got:

```
SHOW CREATE TABLE PRODUCT_COPY;
```

> **The copy did not bring the primary key with it**
>
> Read the output carefully. There is no PRIMARY KEY line. Copying a table this way brings the columns and the rows, and leaves the constraints behind.
>
> So PRODUCT_COPY will happily accept two rows with the same prod_id, which the real PRODUCT table would refuse. A table that looks right and has no constraints is worse than no table, because it quietly collects bad data. Step 13 puts the key back.

Step 13. Section 5.4, change the structure of the copy

ALTER TABLE changes a table that already has rows in it. Three things here: add a column, widen a column, and add the missing constraint.

```
-- add a new column on the end
ALTER TABLE PRODUCT_COPY ADD COLUMN on_sale CHAR(1);

-- make an existing column wider
ALTER TABLE PRODUCT_COPY MODIFY COLUMN prod_name VARCHAR(60) NOT NULL;

-- put back the primary key the copy did not inherit
ALTER TABLE PRODUCT_COPY ADD PRIMARY KEY (prod_id);

DESCRIBE PRODUCT_COPY;
```

Now prove the constraint took effect. This should fail:

```
INSERT INTO PRODUCT_COPY VALUES (1,'Duplicate',9.99,NULL);
```

You get **error 1062, duplicate entry 1 for key PRIMARY**, the same error from step 8. That is the proof. Before step 13 that row would have been accepted.

> **Watch out when you MODIFY a column**
>
> MODIFY COLUMN replaces the whole definition rather than adjusting part of it. If you leave out **NOT NULL** when you only meant to change the width, you have quietly removed the NOT NULL rule and the column will start accepting empty values.
>
> That is why the statement above repeats NOT NULL. Always write out the full definition you want, not just the part you are changing.

Last, remove the column you added:

```
ALTER TABLE PRODUCT_COPY DROP COLUMN on_sale;
```

DROP COLUMN takes the column and everything in it, with no confirmation and no undo. Here it is a column you added a minute ago with nothing in it, so there is nothing to lose, which is the only sort of situation to practise it in.

Step 14. Section 5.5, insert, update and delete a row

Three statements that change data rather than structure. Run them one at a time and look at the row in between, so you can see each one land.

```
-- add a row
INSERT INTO PRODUCT_COPY VALUES (5,'Kenya Peaberry',21.40);
SELECT * FROM PRODUCT_COPY WHERE prod_id = 5;

-- change that row
UPDATE PRODUCT_COPY SET unit_price = 22.95 WHERE prod_id = 5;
SELECT * FROM PRODUCT_COPY WHERE prod_id = 5;

-- remove that row
DELETE FROM PRODUCT_COPY WHERE prod_id = 5;
SELECT COUNT(*) AS rows_left FROM PRODUCT_COPY;
```

> **The WHERE clause is the whole safety net**
>
> Both UPDATE and DELETE above have a WHERE clause. Leave it off an UPDATE and you change **every row in the table**. Leave it off a DELETE and you empty the table.
>
> There is no confirmation prompt and no undo. The habit that saves you: run your WHERE clause as a SELECT first and look at what comes back. If the SELECT returns the rows you meant, the UPDATE or DELETE will hit those same rows.

Step 15. Section 5.6, create a view and update through it

You made a view in the midterm lab and read from it. This time you write through it.

```
CREATE VIEW CHEAP_PRODUCTS AS
SELECT prod_id, prod_name, unit_price
```

```
FROM PRODUCT_COPY
WHERE unit_price < 16.00;

SELECT * FROM CHEAP_PRODUCTS;
```

Two rows come back, the two under 16.00. Now check the real table, change the value **through the view**, and check the real table again:

```
-- before
SELECT prod_id, unit_price FROM PRODUCT_COPY WHERE prod_id = 3;

-- this UPDATE names the VIEW, not the table
UPDATE CHEAP_PRODUCTS SET unit_price = 13.99 WHERE prod_id = 3;

-- after
SELECT prod_id, unit_price FROM PRODUCT_COPY WHERE prod_id = 3;
```

The price goes from 12.60 to 13.99 in **PRODUCT_COPY**, even though your UPDATE never mentioned that table. That is an updatable view: the view holds no data of its own, so a change made through it goes straight to the table underneath.

Not every view works that way. Try this one:

```
CREATE VIEW PRICE_SUMMARY AS
SELECT COUNT(*) AS n, SUM(unit_price) AS total
FROM PRODUCT_COPY;

UPDATE PRICE_SUMMARY SET total = 99;
```

That one is refused with **error 1288, the target table PRICE_SUMMARY of the UPDATE is not updatable**. The view's single row was worked out from four rows, so there is no one row underneath for the database to change. A view you can write through has to point at particular rows, which rules out anything built with an aggregate like COUNT or SUM.

Part 5. Put your file on GitHub

GitHub gives your file a web address. That address is what you hand in, and it is the only thing you hand in, so take your time over these six steps.

Save your file first. Press **Ctrl and S** in VS Code and make sure the OneDrive icon shows a tick.

Step 16. Sign in to GitHub

Open a browser and go to **github.com**.

- If you already have an account, click **Sign in** at the top right.
- If you do not, click **Sign up** and make one. It is free. Use any email address you can read right now, because GitHub sends a code you have to type back in.
- Write your username down. You will need it in step 20.

Step 17. Create the repository

A repository, usually shortened to repo, is one project folder on GitHub.

1. Click the **+** at the very top right of the page, next to your profile picture, then choose **New repository**.
2. In **Repository name**, type **cis4720-lab7**.
3. Leave the description empty.
4. Choose **Public**. This matters more than anything else on the page, see the box below.
5. Tick **Add a README file**.
6. Click the green **Create repository** button at the bottom.

> **Public, not Private**
>
> A private repo cannot be opened by anyone but you. If you choose Private, the address you hand in shows your instructor a page saying the repository does not exist, and the lab reads as not submitted.
>
> If you have already made it private, you can change it: **Settings** on the repo, scroll to the bottom, **Change repository visibility**.
>
> There is nothing private in this file. It is four rows about coffee.

Step 18. Upload your SQL file

You are now looking at your empty repo with just a README in it.

1. Click **Add file**, near the green Code button, then choose **Upload files**.
2. Click **choose your files**, then find **CIS4720_Lab7** inside your OneDrive and pick **lab6_backup.sql**. You can also drag the file onto the page from File Explorer.
3. Wait for the file name to appear on the page. If it is not listed, it did not upload.

Step 19. Commit it

Committing is how GitHub saves a change, along with a note about what the change was.

1. Scroll down to the **Commit changes** box.
2. In the first line, replace the default text with something that says what you did, such as **Add lab7 SQL file with Chapter 5 statements**.
3. Click the green **Commit changes** button.

The page goes back to your repo and **lab6_backup.sql** is now listed. Click it. You should see your statements and your comments. If the comments are missing, you uploaded an older copy of the file, so save in VS Code and upload again.

Step 20. Copy the address, then check it the hard way

The address of your repo is in the browser bar and looks like this, with your own username in place of the example:

```
https://github.com/yourusername/cis4720-lab7
```

Copy it. Then check it properly, because this is the single most common way to lose the marks for this lab:

1. Open a **private** or **incognito** browser window. In most browsers that is **Ctrl, Shift and N**.
2. Paste the address and press Enter.
3. If you can see your repo and your file while signed out, your instructor can too. If you get a **404** page, the repo is still private, so go back and fix it as described in step 17.

Step 21. Hand in the address

Paste the address into the Canvas submission box for this lab and submit. That address is the whole submission. There is no file to upload to Canvas.

> **If you would rather use git on the command line**
>
> You do not have to, and the upload route above is the one to use in the lab. But if you want to see how it is really done, Git has to be installed and you will need a personal access token instead of your password:

```
cd "C:\Users\yourname\OneDrive\CIS4720_Lab7"
git init
git add lab6_backup.sql
git commit -m "Add lab7 SQL file"
git branch -M main
git remote add origin https://github.com/yourusername/cis4720-lab7.git
git push -u origin main
```

Worth knowing why this is the harder path on a lab PC: your git settings and saved credentials live on the machine, and the machine is wiped tonight, so you would set all of it up again next week. The browser upload has nothing stored locally to lose.

How this is graded

Steps	What it shows	Points
1 to 3	Signed in to the OneDrive app, folder made on the PC, file in it	1.0
4 to 6	Connected from VS Code, and you can comment a line and a block	1.0
7 to 9	The file ran, and CREATE and INSERT are commented out, not deleted	2.0
10 to 15	All six Chapter 5 statements present and working	3.0
all	A comment in your own words above every statement you added	1.0
16 to 21	A public GitHub repo holding your file, and the address handed in	2.0
	Total	10.0

Your repository is the evidence for all of it, which is why the address is the only thing you hand in. A statement that is not in the uploaded file did not happen as far as grading goes, so upload the file after you finish the SQL and not before. The due date is listed on Canvas LMS.

Previous labs: in_class_lab6_instructions.pdf and in_class_lab_midterm.pdf.

Reference: Coronel, C. and Morris, S. (2027). Database Systems: Design, Implementation, and Management (14th ed.). Cengage Learning. ISBN 979-8214410241. Chapter 5, sections 5.1 to 5.6.